

# APACHE KAFKA

FUNDAMENTALS



By Michael Fernandes



# TABLE OF CONTENTS

1

## Introduction to Kafka

- Limitations of Traditional Messaging Systems
- What is Apache Kafka
- Features of Apache kafka
- Kafka vs Traditional Messaging Systems

2

## Kafka Architecture

- Kafka Architecture
- Kafka Components

3

## Kafka Core Concepts

- Topics and Partitions
- Offsets
- Replication
- Retention
- Log Compaction

4

## Kafka Producers

- Kafka Producer Workflow
- Producer Configurations
- Delivery Semantics

5

## Kafka Consumers

- Kafka Consumer Workflow
- Consumer Groups
- Rebalancing

6

## Kafka Internals

- Kafka Storage Internals
- Leader and Follower Replication

7

## Kafka Stream Processing

- What is Kafka Streams
- Streaming Workflow
- Streaming Use Cases

8

## Kafka Connect

- What is Kafka Connect
- Kafka Connect Workflow
- Connectors

9

## Kafka Installation and Ecosystem Integration

Kafka+ Hadoop  
Spark + Kafka

10

## Kafka Use Cases

# Part I - Introduction to Kafka

## 1.1

### Limitations Of Traditional Messaging Systems

#### High Latency

Traditional systems struggle with:

- Real-time processing
- Large-scale event streams
- Continuous data ingestion

Why This Is a Problem

- Slow data delivery
- Delayed analytics
- Poor streaming performance

#### Limited Scalability

Many legacy messaging systems:

- Scale vertically
- Cannot efficiently handle massive throughput
- Become bottlenecks under heavy load

Result

Difficult to support:

- Millions of events/sec
- Distributed applications
- Large streaming pipelines

#### Limited Stream Processing

Older systems were mainly designed for:

- Simple message queues
- Request-response communication

Not Ideal For

- Event-driven systems
- Real-time analytics
- Streaming ETL pipelines
- Live dashboards

01

02

03

04

05

#### Inefficient Data Retention

Traditional queues usually:

- Remove messages after consumption
- Do not maintain long-term logs

Problems

- No replay capability
- Difficult debugging
- Limited event tracking

#### Weak Fault Tolerance

Failures may cause:

- Message loss
- Downtime
- Inconsistent data delivery

Challenges

- Limited replication
- Poor recovery mechanisms
- Single point of failure



Apache Kafka is a:

- Distributed event streaming platform
- High-throughput messaging system
- Fault-tolerant data pipeline engine
- Real-time stream processing platform

Kafka is designed for:

- Distributed computing
- Real-time analytics
- Event-driven architectures
- Massive-scale data streaming

### Distributed Architecture

Kafka distributes: Data,  
Partitions and Processing  
across multiple servers.

### Fault Tolerance

Kafka uses:

- Replication
- Leader-follower architecture
- Automatic recovery  
to prevent data loss.

### High Throughput

Kafka handles:

- Massive event streams
- Millions of messages/sec
- Large-scale pipelines

### Big Data Ecosystem Integration

Kafka integrates with:

- Spark
- Hadoop
- Hive
- Airflow
- Flink
- Databases
- Cloud platforms

### Persistent Storage

Messages are stored:

- On disk
- Across partitions
- Using distributed logs

### Real-Time Streaming

Kafka supports:

- Continuous event ingestion
- Real-time analytics
- Streaming pipelines

### Scalability

Kafka can scale:

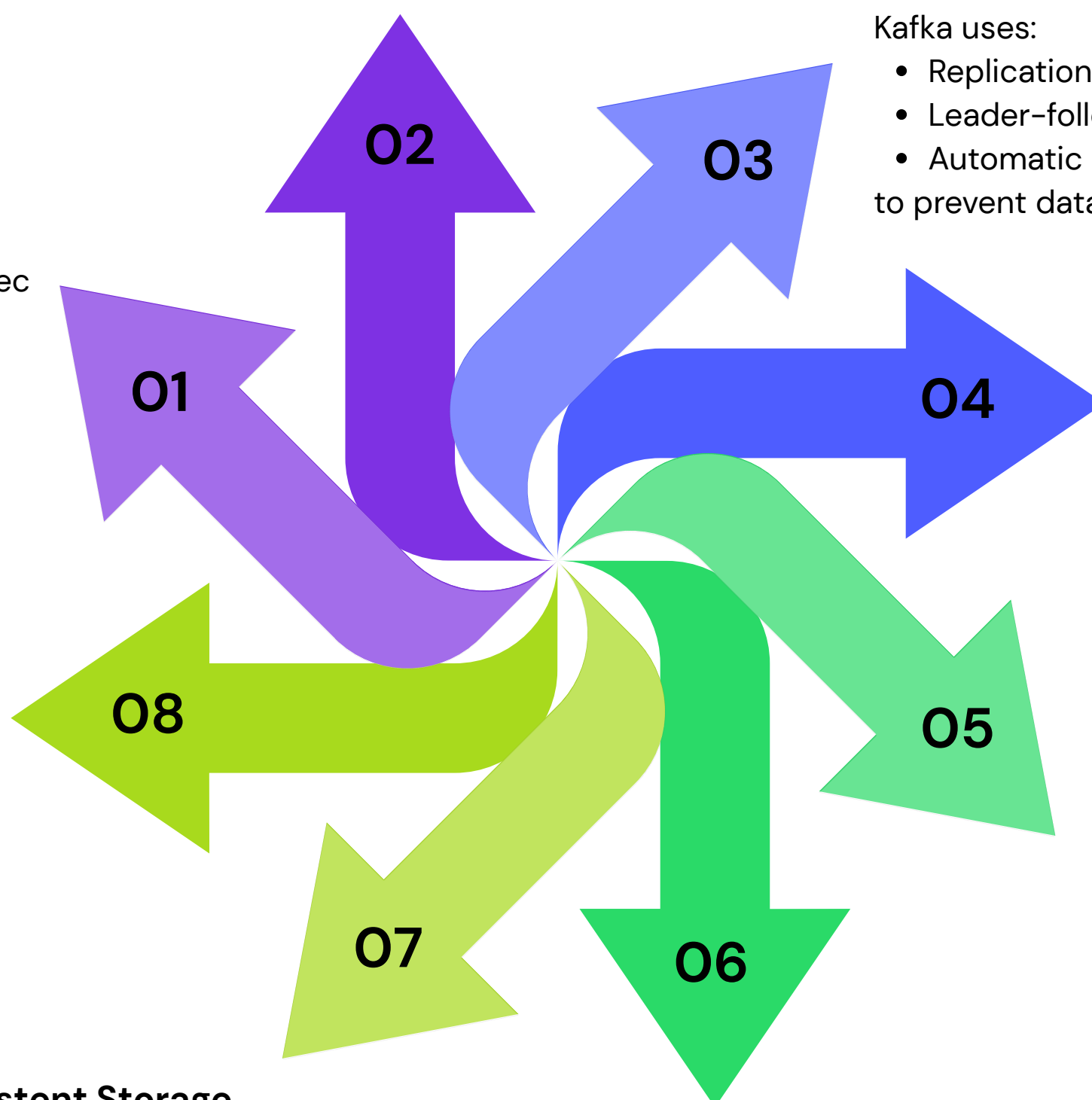
- From small clusters
- To enterprise-scale  
distributed systems  
by adding brokers.

### Event Replay

Kafka retains events for  
configurable durations.

Benefits:

- Event reprocessing
- Debugging
- Recovery
- Audit tracking





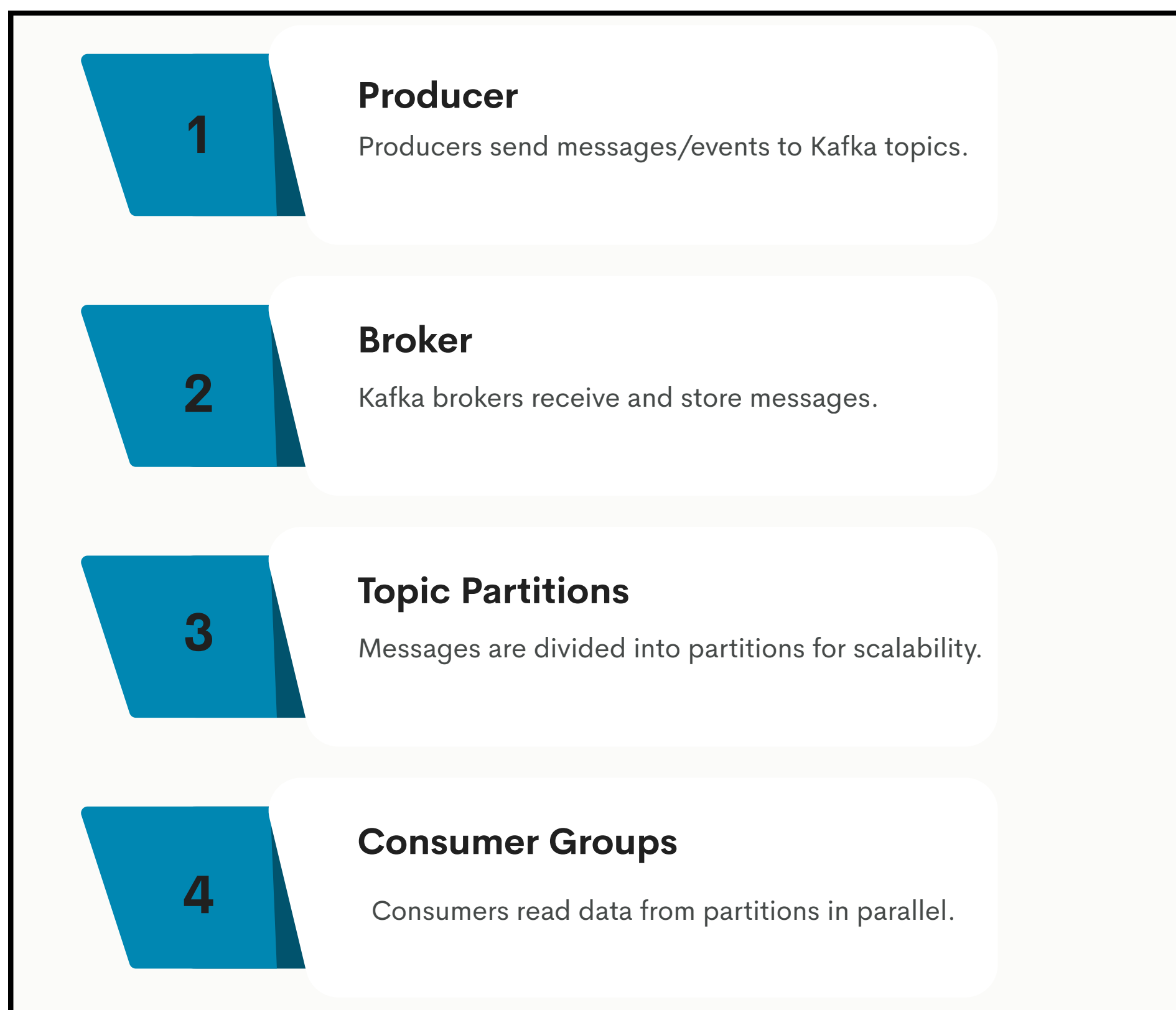
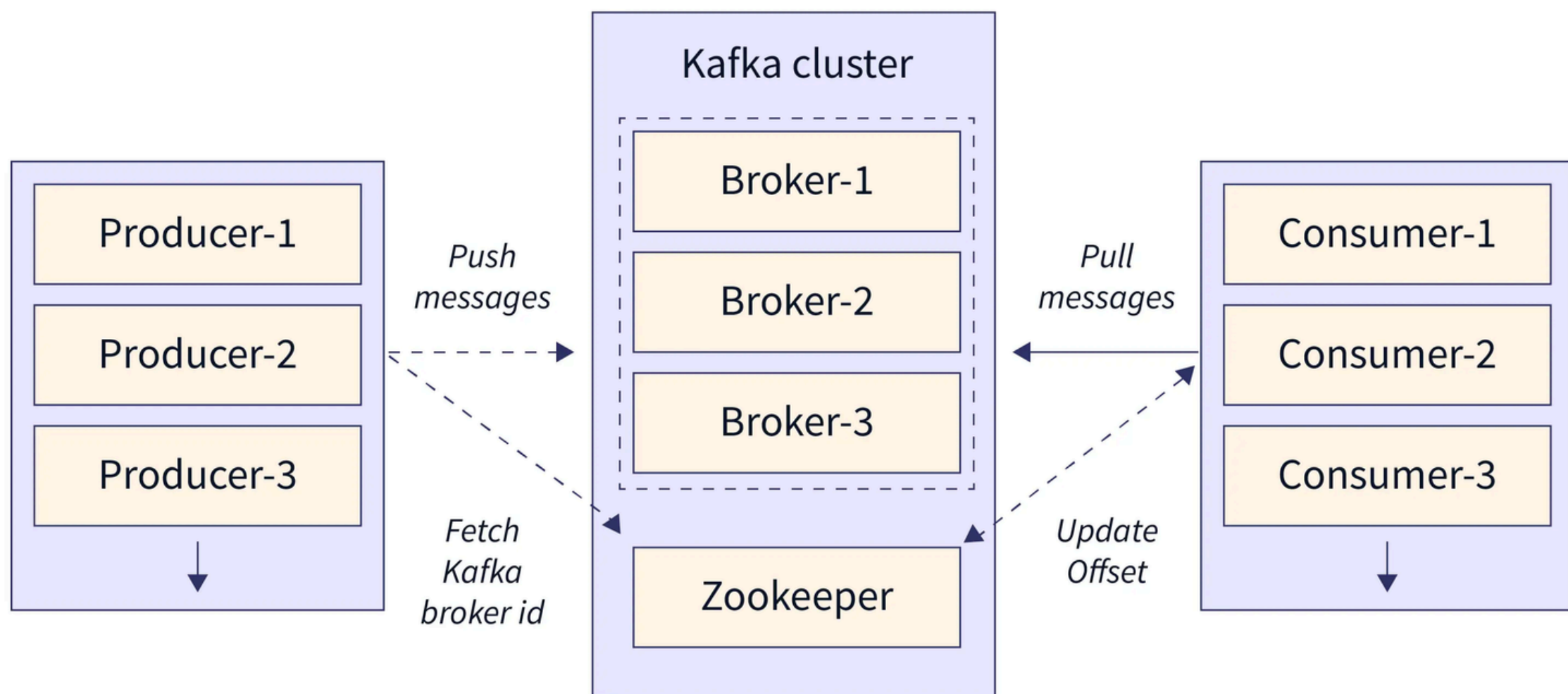
| Feature              | Traditional Messaging | Kafka                    |
|----------------------|-----------------------|--------------------------|
| Processing Model     | Queue Based           | Distributed Streaming    |
| Scalability          | Limited               | Highly Scalable          |
| Fault Tolerance      | Moderate              | Strong Replication       |
| Message Storage      | Temporary             | Persistent Storage       |
| Throughput           | Lower                 | Very High                |
| Real Time Streaming  | Limited               | Excellent                |
| Event Replay         | Not Supported         | Fully Supported          |
| Architecture         | Centralized           | Distributed              |
| Parallel Processing  | Limited               | Partition based          |
| Big Data Integration | Limited               | Strong Ecosystem Support |



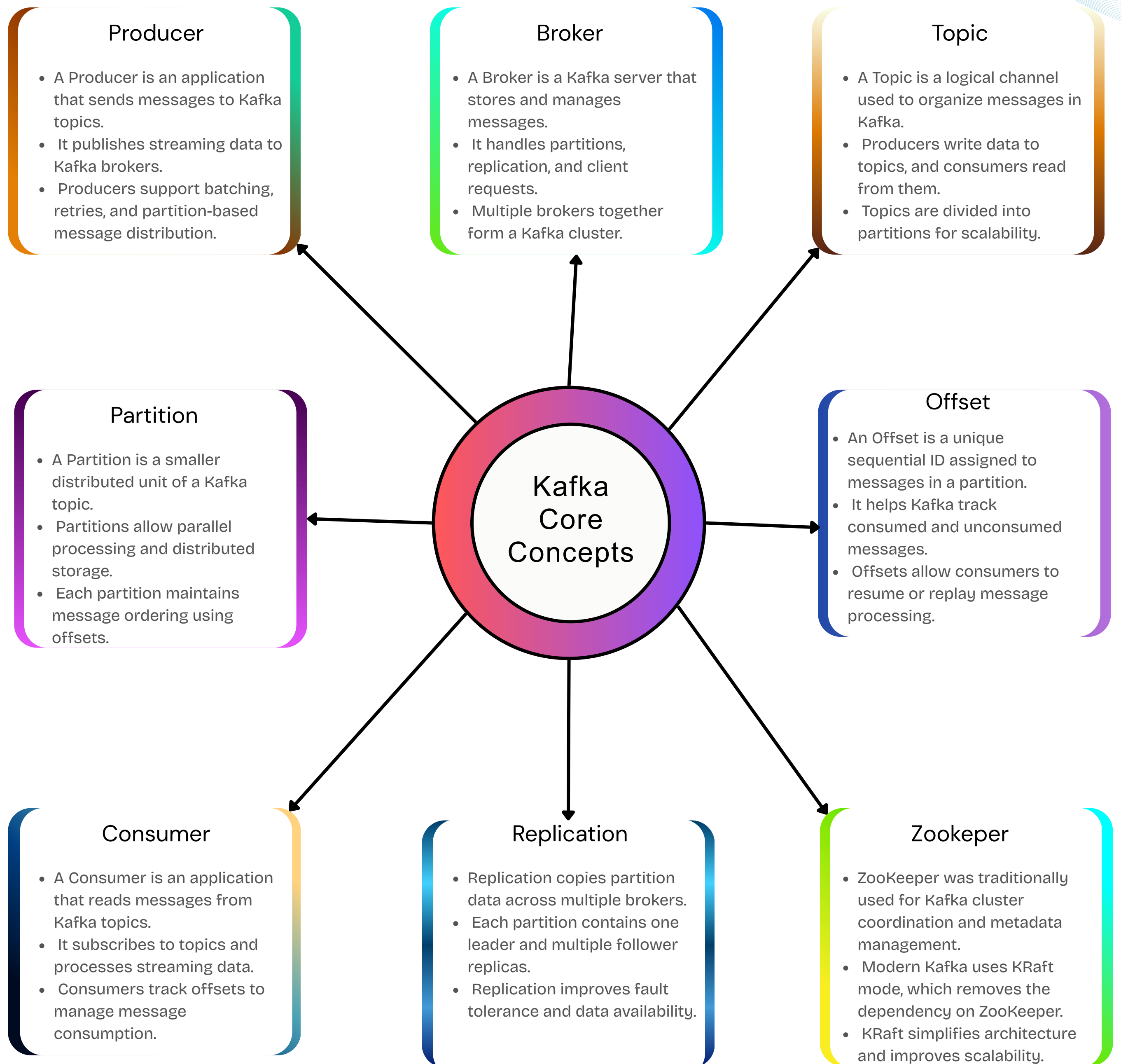
# Part II - Kafka Architecture

2.1

## Kafka Architecture







# Part III - Kafka Core Concepts

## 3.1

## Topics and Partition

### Topics

A Topic is a logical category used to store messages in Kafka.

Producers send data to topics, and consumers read data from them.

Examples include payments, orders, logs, and transactions.

### Partitions

A Partition is a smaller distributed unit of a topic.

Kafka divides topics into partitions for parallel processing and scalability.

Each partition stores messages in sequential order.

### Why Partitions are Important

#### 1. Parallel Processing

Multiple consumers can process partitions simultaneously.

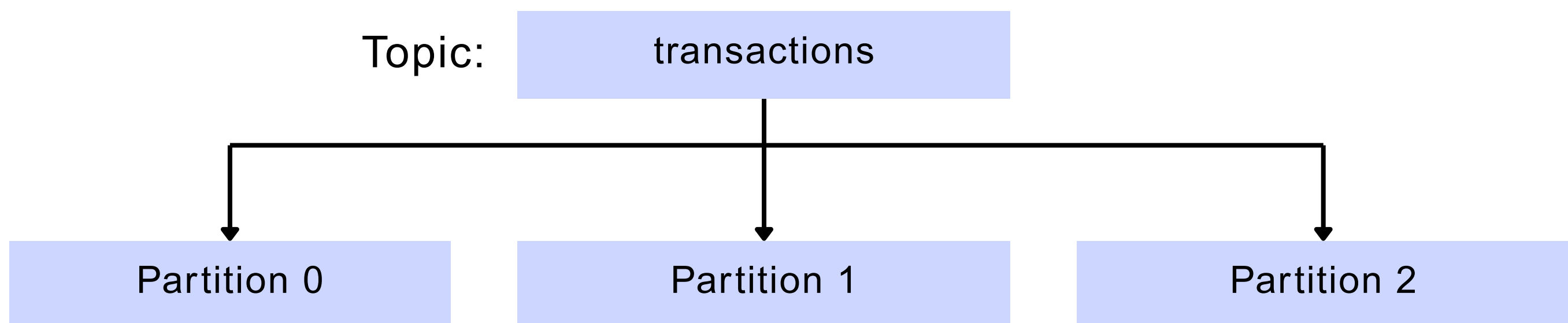
#### 2. Scalability

Partitions distribute workload across brokers.

#### 3. High Throughput

More partitions increase processing capacity.

### Example



Messages are distributed across these partitions.



An Offset is a unique sequential identifier assigned to each message inside a partition.

Offsets help Kafka track message consumption progress.

Each partition maintains its own offset sequence.

### Offset Management

Kafka supports:

- Automatic offset commits
- Manual offset commits

Manual commits provide better control over processing reliability.

### Why Offsets are Important

#### 1. Message Tracking

Consumers know which messages are processed.

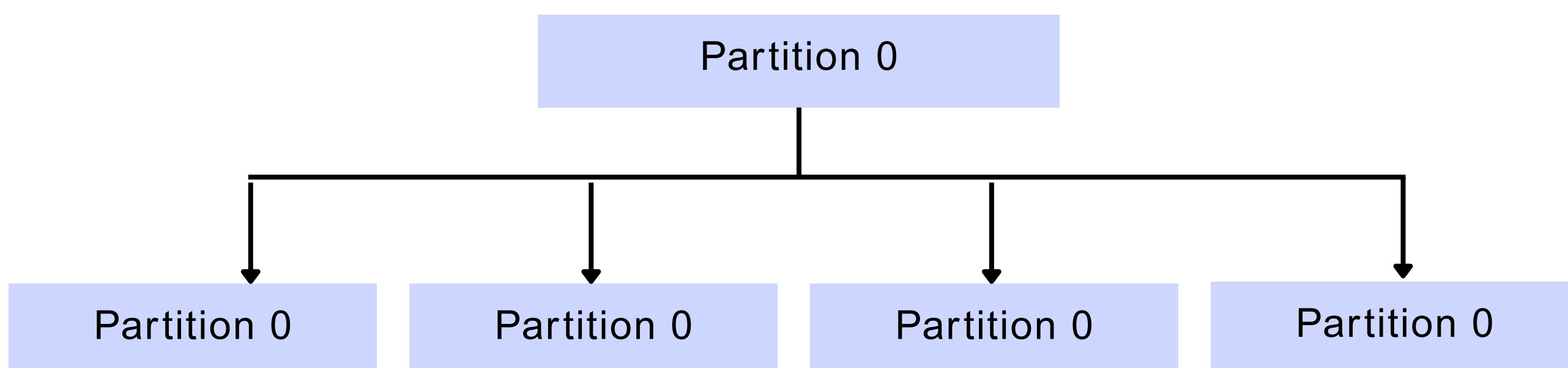
#### 2. Fault Recovery

Consumers can resume from the last committed offset.

#### 3. Event Replay

Kafka can re-read older messages when needed.

### Example



Each new message receives the next offset value.

### 3.3

## Replication

Replication copies partition data across multiple brokers.

It protects Kafka from broker failures and data loss.

Each partition has one leader and multiple follower replicas.

### Benefits of Replication

#### 1. Fault Tolerance

Protects against broker failures.

#### 2. High Availability

Kafka remains operational during failures.

#### 3. Data Durability

Multiple copies reduce risk of data loss.

### Leader Replica

The Leader handles:

- Read requests
- Write requests
- Client communication

All producers and consumers interact with the leader.

### Follower Replica

Followers copy data from the leader continuously.

If the leader fails, one follower becomes the new leader.

This ensures high availability and fault tolerance.

### 3.4

## Retention

Retention defines how long Kafka stores messages.

Messages remain available even after consumers read them.

Kafka stores data based on configurable retention policies.

#### 1. Time-Based Retention

Messages are deleted after a specific duration.  
Example: 7 days retention

#### 2. Size-Based Retention

Messages are deleted when storage exceeds a configured size.  
Example: 100 GB retention limit



## Why Retention is Important

### 1. Event Replay

Consumers can reprocess older events.

### 2. Debugging

Historical events remain available.

### 3. Recovery

Applications can recover missed data

## Retention Example

Kafka may store:

- Logs
- Transactions
- Streaming events for several days before deletion.

## 3.5

## Log Compaction

Log Compaction keeps only the latest value for each message key.

Older records with the same key are removed gradually.

This reduces storage while preserving the latest state

## Why Log Compaction is Important

### 1. Storage Optimization

Reduces unnecessary old records.

### 2. Latest State Recovery

Consumers can rebuild current application state.

### 3. Event Sourcing

Useful for databases and streaming systems.

## Example

Messages:

- User1 → Active
- User1 → Inactive
- User1 → Premium

After compaction:

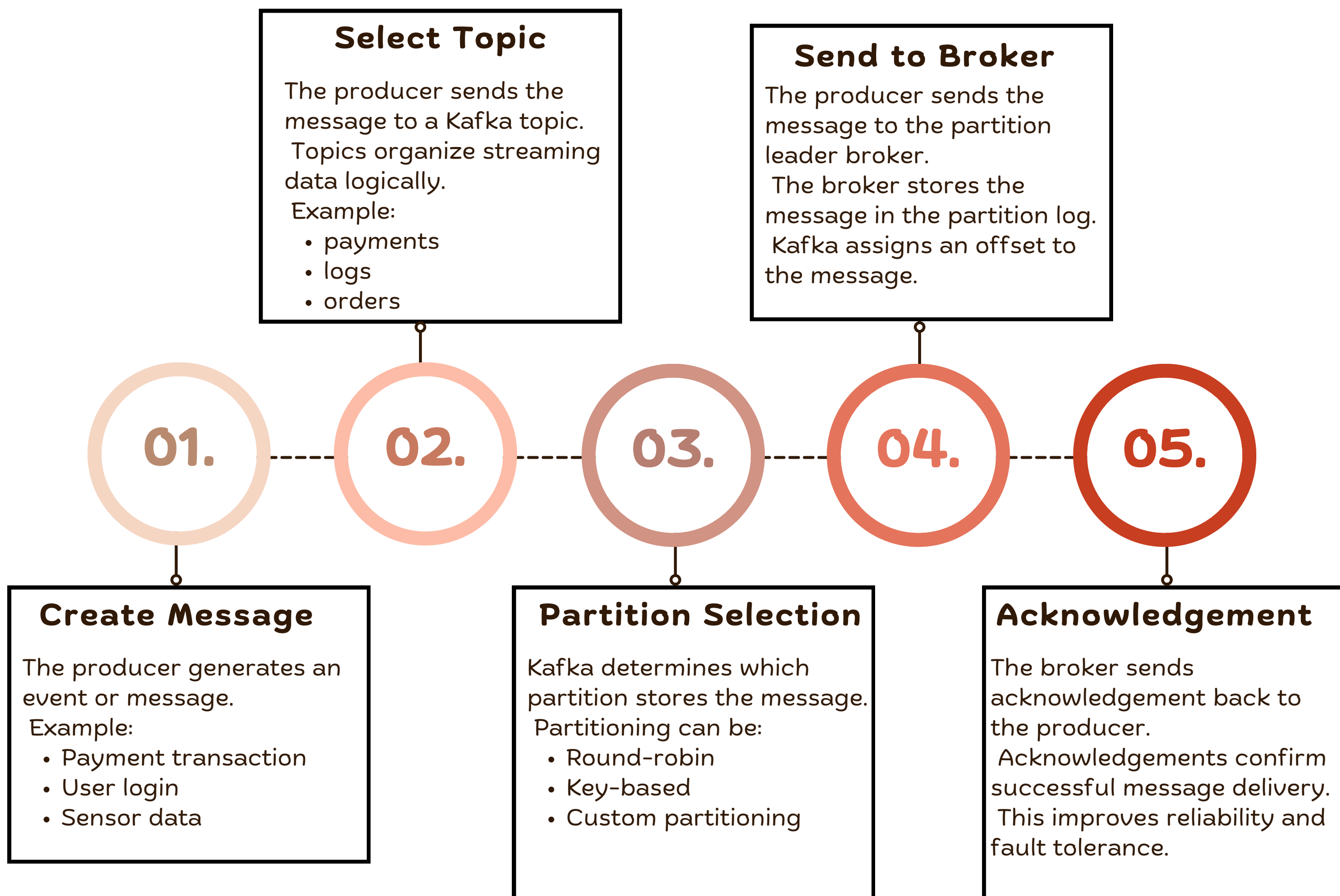
User1 → Premium

Only the latest value is retained.

# Part IV - Kafka Producers

## 4.1

### Kafka Producers Workflow





### 1. acks

The acks configuration controls acknowledgement behavior.

#### Types

acks = 0

Producer does not wait for acknowledgement.

Fastest but may lose data.

acks = 1

Leader broker confirms message receipt.  
Balanced performance and reliability.

acks = all

All replicas confirm the message.  
Highest reliability and strongest durability.

### 2. retries

Defines how many times the producer retries failed requests.  
Retries improve fault tolerance during temporary failures.

Higher retries reduce risk of message loss.

### 3. batch.size

Controls the number of messages grouped together before sending.

Larger batches improve throughput.

Very large batches may increase latency.

### 4. compression.type

Compresses messages before transmission.

#### Common Types

- gzip
- snappy
- lz4
- zstd

Compression reduces:

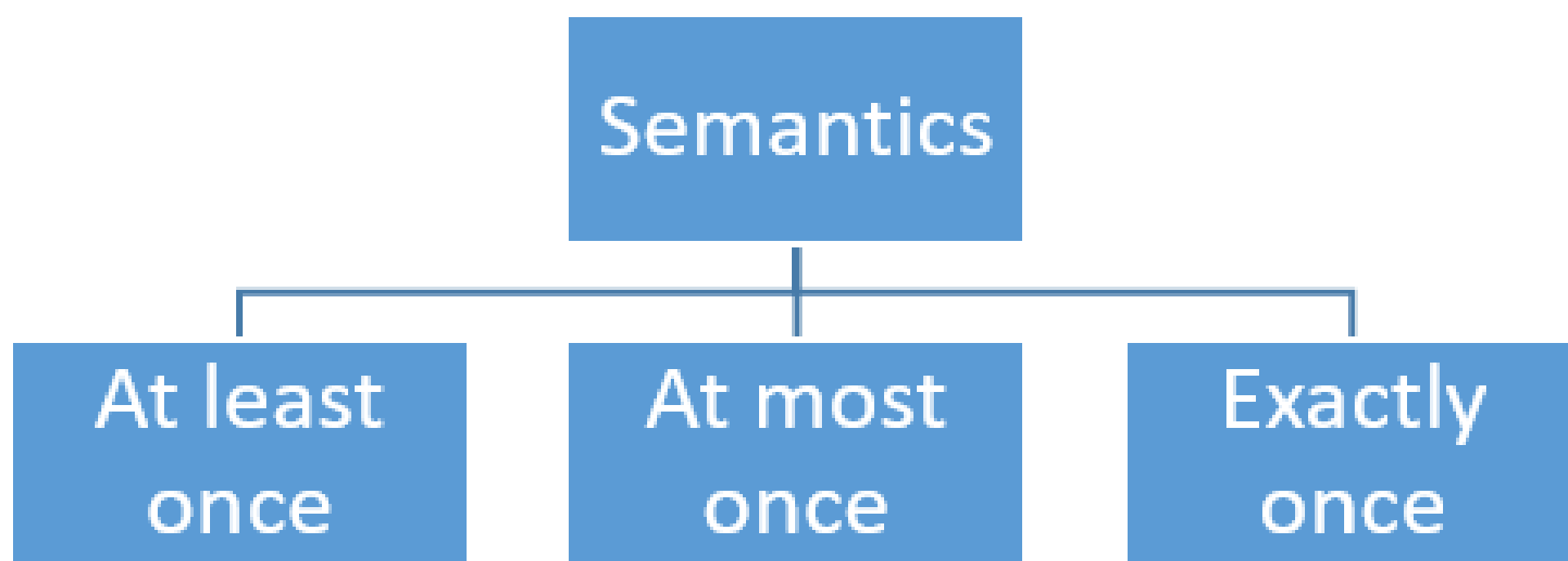
- Network traffic
- Storage usage

### 5. linger.ms

Defines how long the producer waits before sending batches.

Allows more messages to accumulate in a batch.

Improves batching efficiency and compression.



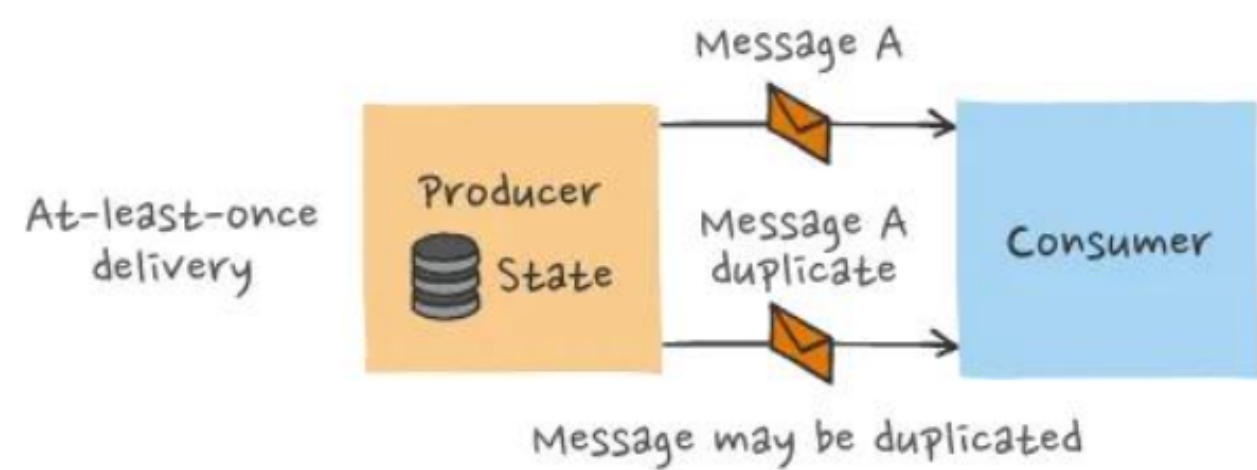
### 1. At-least-once delivery

Messages are delivered zero or one time.

Message is never lost but may be duplicated

Characteristics

- Fastest delivery
- Lowest reliability



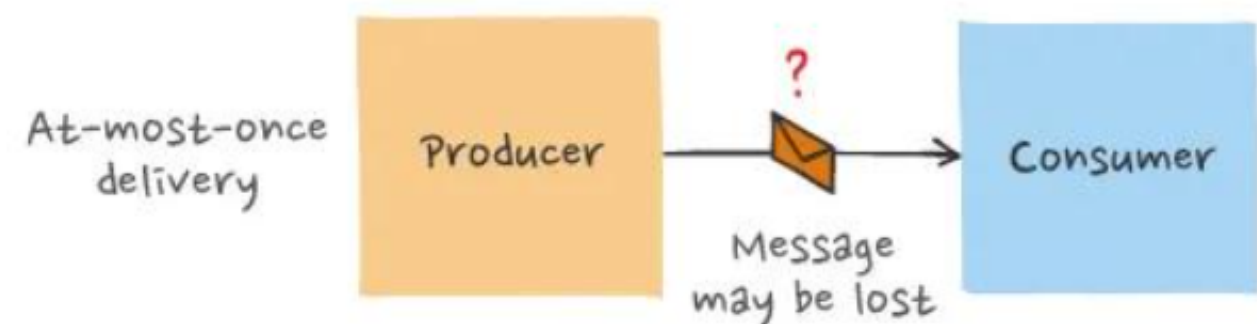
### 2. At-most-once delivery

Messages are delivered one or more times.

Message may be lost but never duplicated

Characteristics

- Reliable delivery
- Possible duplicate events



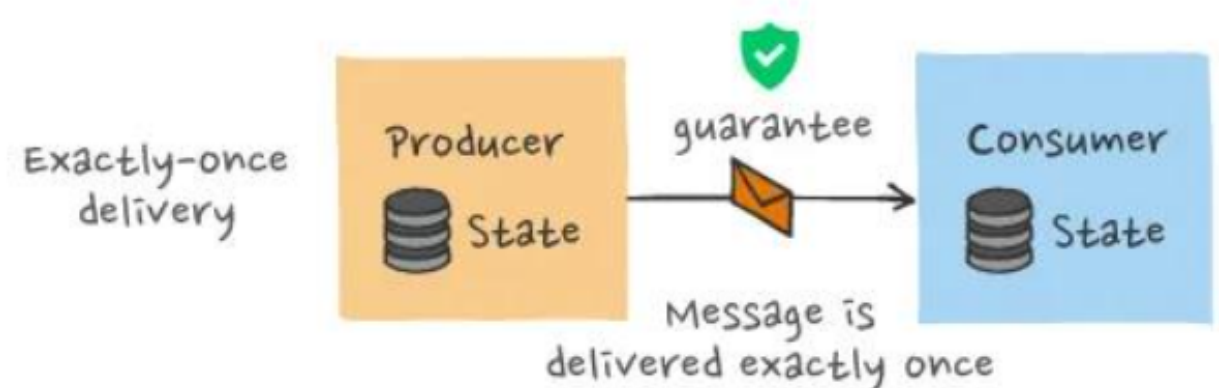
### 3. Exactly once delivery

Messages are delivered exactly one time without duplicates.

Kafka uses idempotent producers and transactions.

Characteristics

- No duplicates
- No message loss





# Part V - Kafka Consumers

## 5.1

### Kafka Consumers Workflow

#### Subscribe to Topic

The consumer subscribes to one or more Kafka topics.

Example:

- payments
- logs
- transactions

#### Read Partition Data

Consumers read messages using offsets.

Offsets track the current reading position inside partitions.

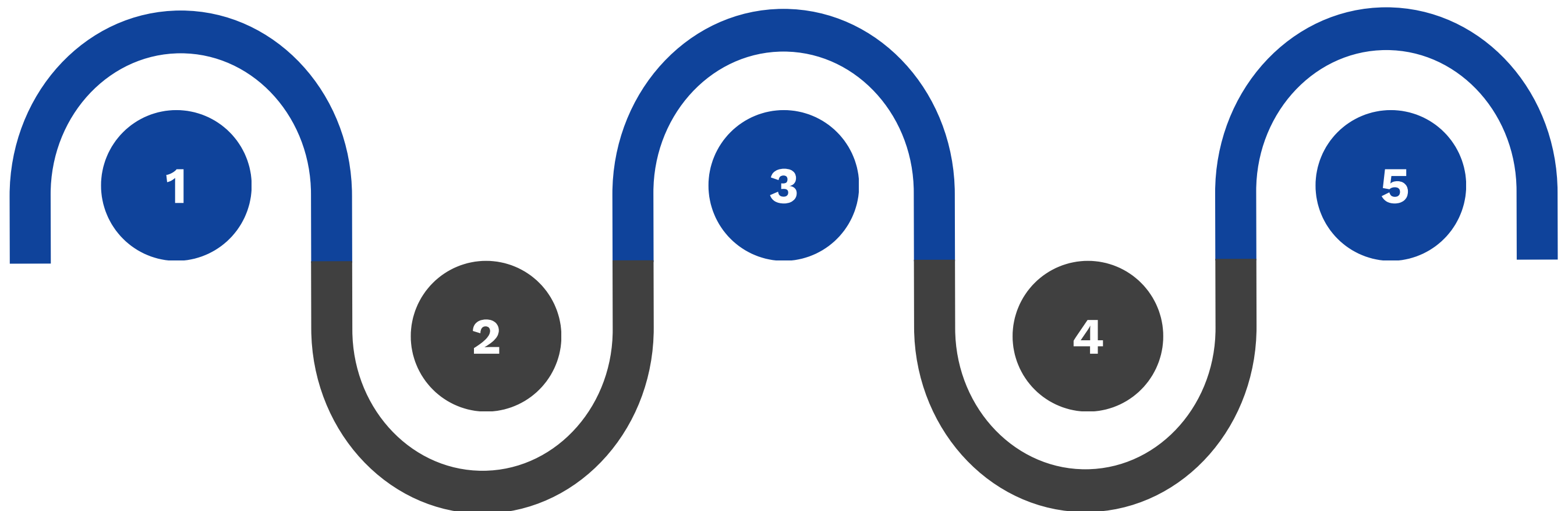
Each consumer maintains its own offset progress.

#### Commit Offsets

After processing, the consumer commits offsets to Kafka.

This helps Kafka track consumed messages.

Consumers can resume from the last committed offset after failures.



#### Poll Messages

Consumers read messages using offsets.

Offsets track the current reading position inside partitions.

Each consumer maintains its own offset progress.

#### Process Messages

The consumer processes incoming events.

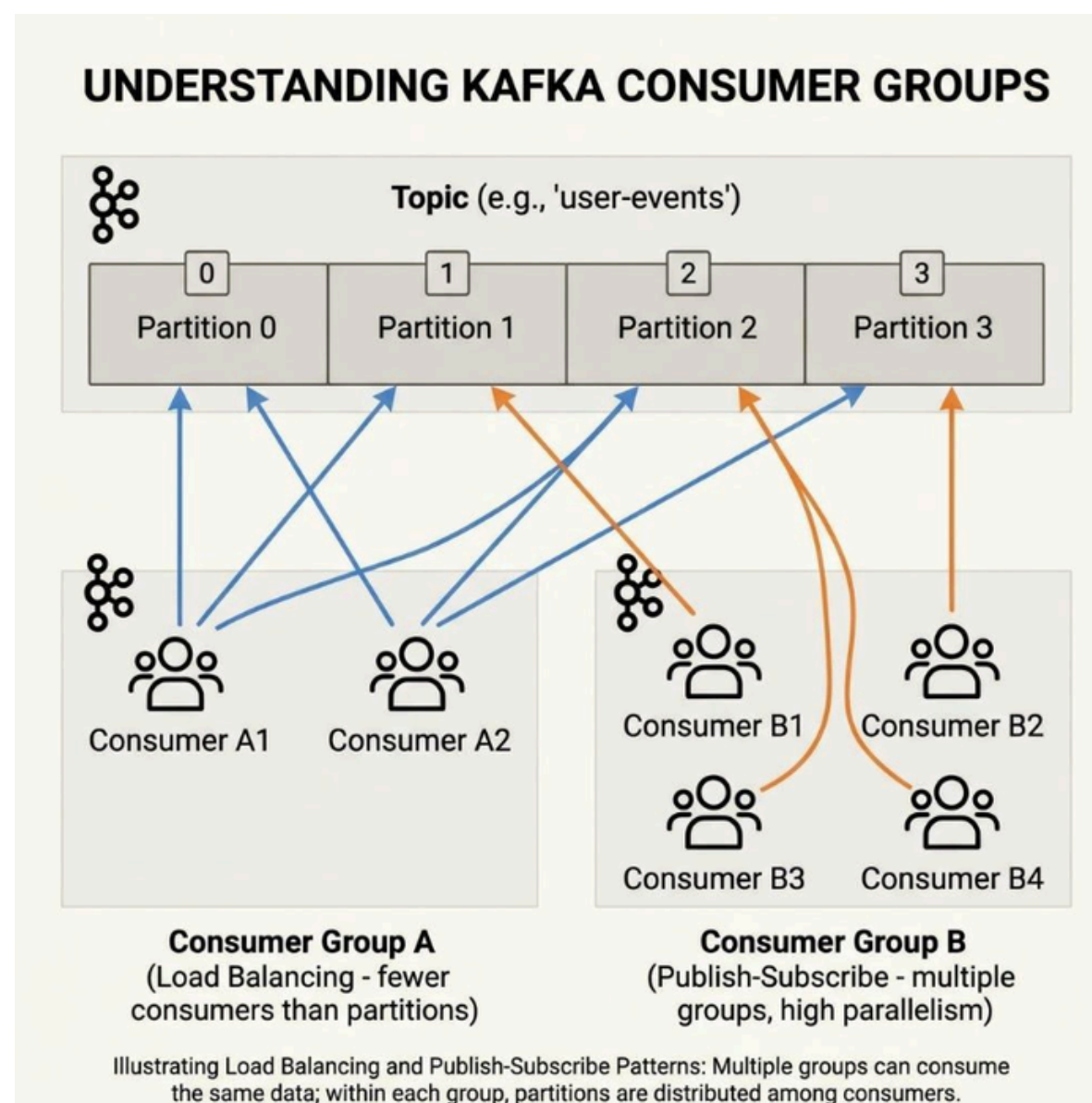
Examples:

- Fraud detection
- Real-time analytics
- Log processing

A Consumer Group is a collection of consumers working together to process data from a topic.

Kafka distributes partitions among consumers within the group.

This enables scalable and parallel message consumption.



### How Consumer Groups Work

Example:

Topic → 4 Partitions

Consumer Group:

- Consumer 1 → Partition 0 & 1
- Consumer 2 → Partition 2 & 3

Each partition is consumed by only one consumer within a group.

Inside a consumer group:

One partition → One consumer only

However:

One consumer can read multiple partitions.

### Benefits of Consumer Groups

#### 1. Parallel Processing

Multiple consumers process partitions simultaneously.

#### 2. Scalability

Adding consumers improves processing capacity.

#### 3. Fault Tolerance

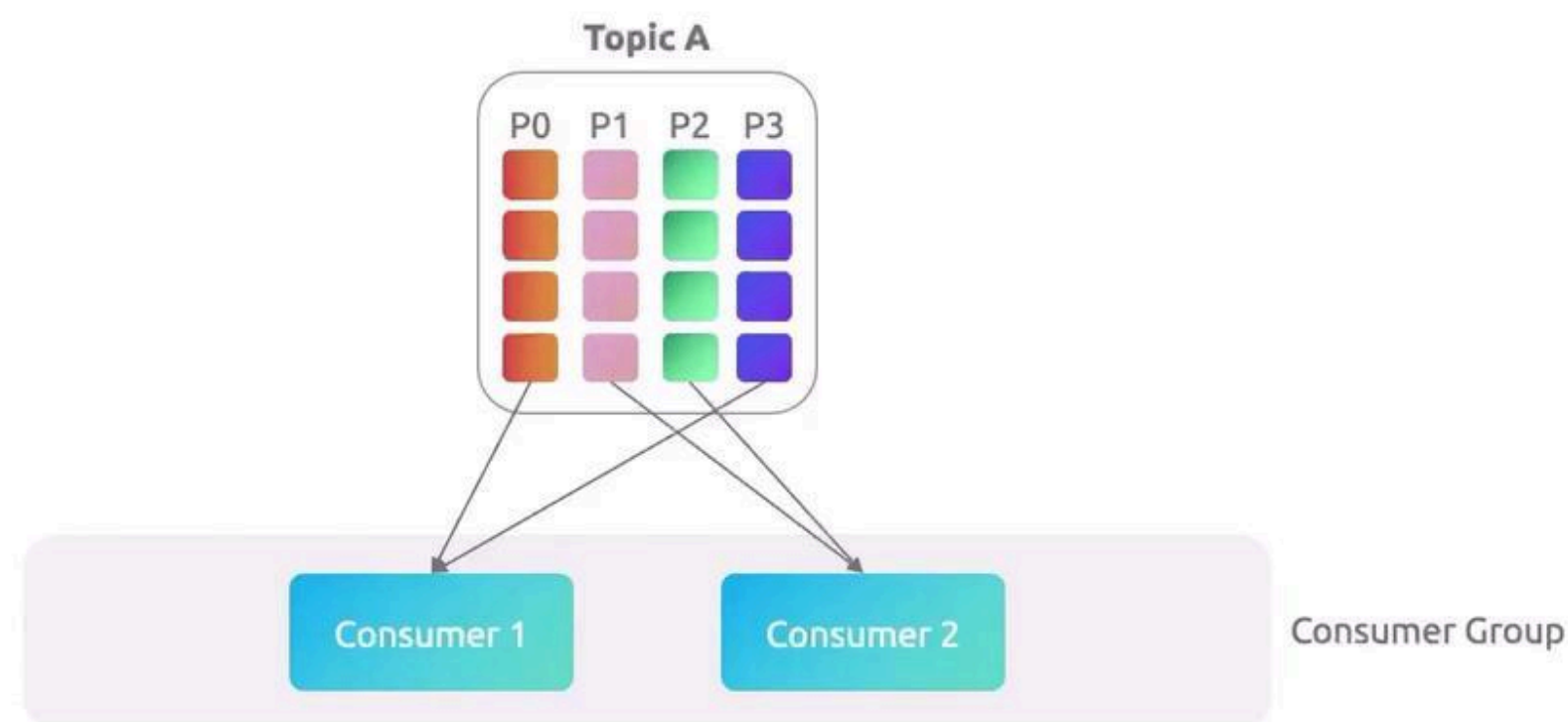
If one consumer fails, partitions are reassigned automatically.

#### 4. Load Balancing

Kafka distributes partitions evenly across consumers.



Rebalancing is the process of redistributing partitions among consumers in a group. Kafka performs rebalancing automatically when group membership changes. This ensures balanced workload distribution.



When consumers join or leave a consumer group, Kafka rebalances partitions among the remaining consumers. Offset plays a vital role in rebalancing.

### When Rebalancing Happens

1. New Consumer Joins  
Partitions are redistributed across consumers.
2. Consumer Fails  
Kafka reassigns partitions to active consumers.
3. Consumer Leaves  
Remaining consumers receive additional partitions.
4. Topic Partitions Increase  
New partitions are assigned across the group.

### Rebalancing Workflow

1. Consumer group pauses processing
2. Kafka reassigns partitions
3. Consumers receive new assignments
4. Processing resumes

### Why Rebalancing is Important

#### Load Balancing

Ensures equal workload distribution.

#### Fault Recovery

Failed consumer partitions are reassigned automatically.

#### Scalability

Supports dynamic scaling of consumers.

# Part VI - Kafka Internals

## 6.1

## Kafka Storage Internals

In distributed streaming systems, storage architecture plays a critical role in performance, scalability, and fault tolerance. Apache Kafka is designed around a highly optimized distributed commit log architecture that enables fast sequential writes, durable message persistence, and efficient message retrieval.

Kafka internally stores topic data inside brokers using partitions, segments, log files, index files, and replication mechanisms. Understanding these storage internals is essential for designing scalable data engineering pipelines.

### Topics and Partitions

Topics are divided into partitions for:

- Scalability
- Parallelism
- Fault tolerance

Example :

Topic: stock-prices

Partition-0

Partition-1

Partition-2

### Offsets

Every message gets a unique sequential ID called an offset.

Example:

Offset 0 → Message A

Offset 1 → Message B

Offset 2 → Message C

Offsets help consumers:

- Track progress
- Resume processing
- Replay messages

### Log Segments

Partitions are divided into smaller units called segments.

Example:

000000000000000000000000.log

000000000000000000000000.index

000000000000000000000000.timeindex

Segment filename indicates base offset.

Example:

000000000000000000000000.\* → Offsets 0–34

000000000000000000000035.\* → Offsets 35+



**.log File**  
Stores actual message data.  
Example:  
  
Offset 0 → BMW  
Offset 1 → Chevrolet  
Offset 2 → Porsche  
  
Kafka writes data sequentially for high throughput.

**.timeindex File**  
Stores mapping:  
  
Timestamp → Offset  
  
Used for:  

- Time-based searches
- Retention management
- Replay by timestamp

**.index File**  
Stores mapping:  
  
Offset → Position in .log file  
  
Example:  
offset: 20 position: 346  
offset: 34 position: 692  
  
Helps Kafka quickly locate messages.

## Retention Policy

Kafka deletes old segments automatically.

Time-Based

log.retention.hours=168

Size-Based

log.retention.bytes=1073741824

## Log Compaction

Keeps only latest value per key.

Example:

Key=A Value=1

Key=A Value=2

After compaction:

Key=A Value=2

Useful for:

- CDC
- State recovery
- User profile updates

# Part VII - Kafka Stream Processing

## 7.1

### What is Kafka Streams

#### What is Kafka Streams

Kafka Streams is a lightweight Java library used for building:

- Real-time data pipelines
- Stream processing applications
- Event-driven systems

It processes data directly from Kafka topics and writes results back to Kafka topics.

Kafka Streams turns Kafka into a real-time processing engine.  
Instead of storing data first and processing later:

Traditional System: Store → Process Later

Kafka Streams processes data immediately:

Kafka Streams: Receive Event → Process Instantly

#### Real-World Example

Suppose a stock market application continuously receives stock prices.

AAPL : 245

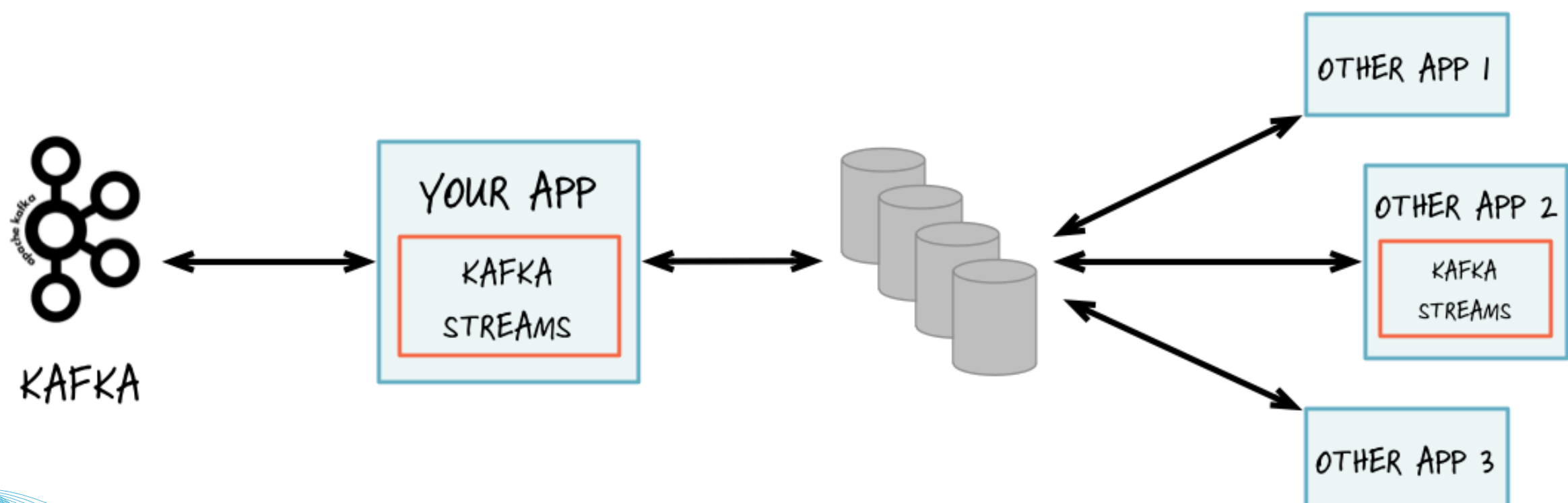
GOOGLE : 180

TSLA : 310

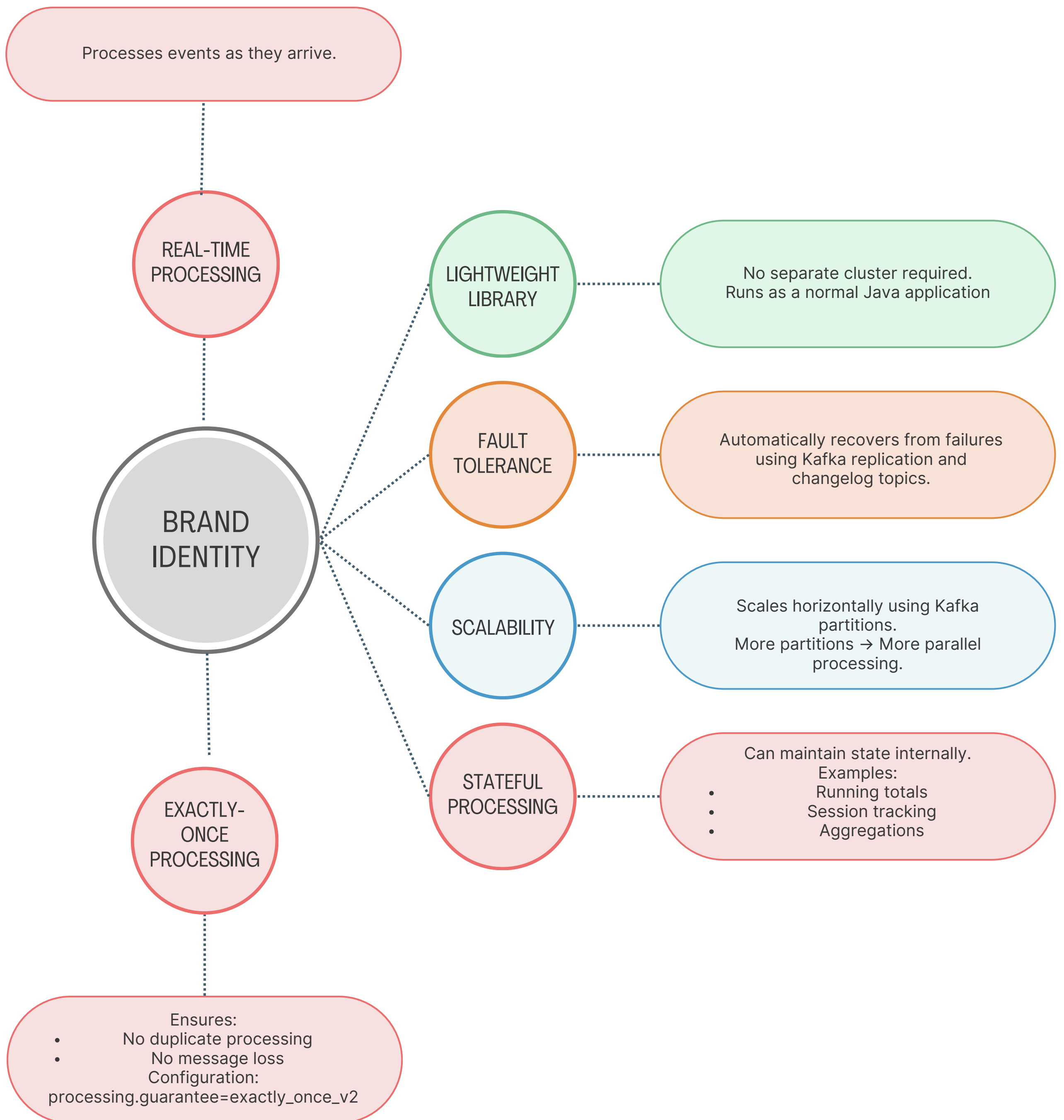
Kafka Streams can:

- Filter expensive stocks
- Calculate moving averages
- Detect trends
- Generate alerts in real time

- 1 Capture business events in Kafka
- 2 Process the events with Kafka Streams
- 3 Must use external DBs and systems to share latest results
- 4 Other apps query DBs for latest results



# Kafka Streams Features





## Main Components

### KStream

Represents continuous event streams.

Example:

Stock Prices Stream  
User Click Stream  
Transaction Stream

### KTable

Represents latest state of data.

Example:

UserID → Current  
Balance

### Topology

Processing pipeline of operations.

Example:

Source → Filter →  
Aggregate → Sink

## Example



```
StreamsBuilder builder = new  
StreamsBuilder();  
  
KStream<String, String> stream =  
builder.stream("stock-prices");  
  
stream.filter((k,v) ->  
v.contains("AAPL"))  
.to("apple-stock-topic");
```

This application:

Reads stock data

Filters Apple stock events

Writes result to another topic

## 1. DATA GENERATION

Data is continuously generated from sources such as:

- Applications
- Sensors
- Websites
- Banking systems

## 2. PRODUCER SENDS EVENTS

Kafka producers publish events into Kafka topics.

Example: `producer.send("stock-prices", data)`

Producer responsibilities include Serialize data ,Select partition and Send records to broker

## 3. EVENTS STORED IN KAFKA TOPICS

Kafka brokers store events inside partitions. Messages are stored sequentially using offsets.

Topic: stock-prices

Partition-0

Partition-1

Partition-2

## 4. KAFKA STREAMS CONSUMES DATA

Kafka Streams application continuously reads records from input topics.

Example: `builder.stream("stock-prices");`

Kafka Streams acts as the processing engine.

## 5. STREAM PROCESSING

Kafka Streams performs transformations and analytics.

Operations include Filter, Map, Aggregate, Join, Windowing

## 6. STATEFUL PROCESSING

Kafka Streams can maintain state internally.

Examples:

- Running totals
- Session tracking

State is stored using local state stores and changelog topics.

## 7. OUTPUT GENERATION

Processed data is written into output topics.

Example:

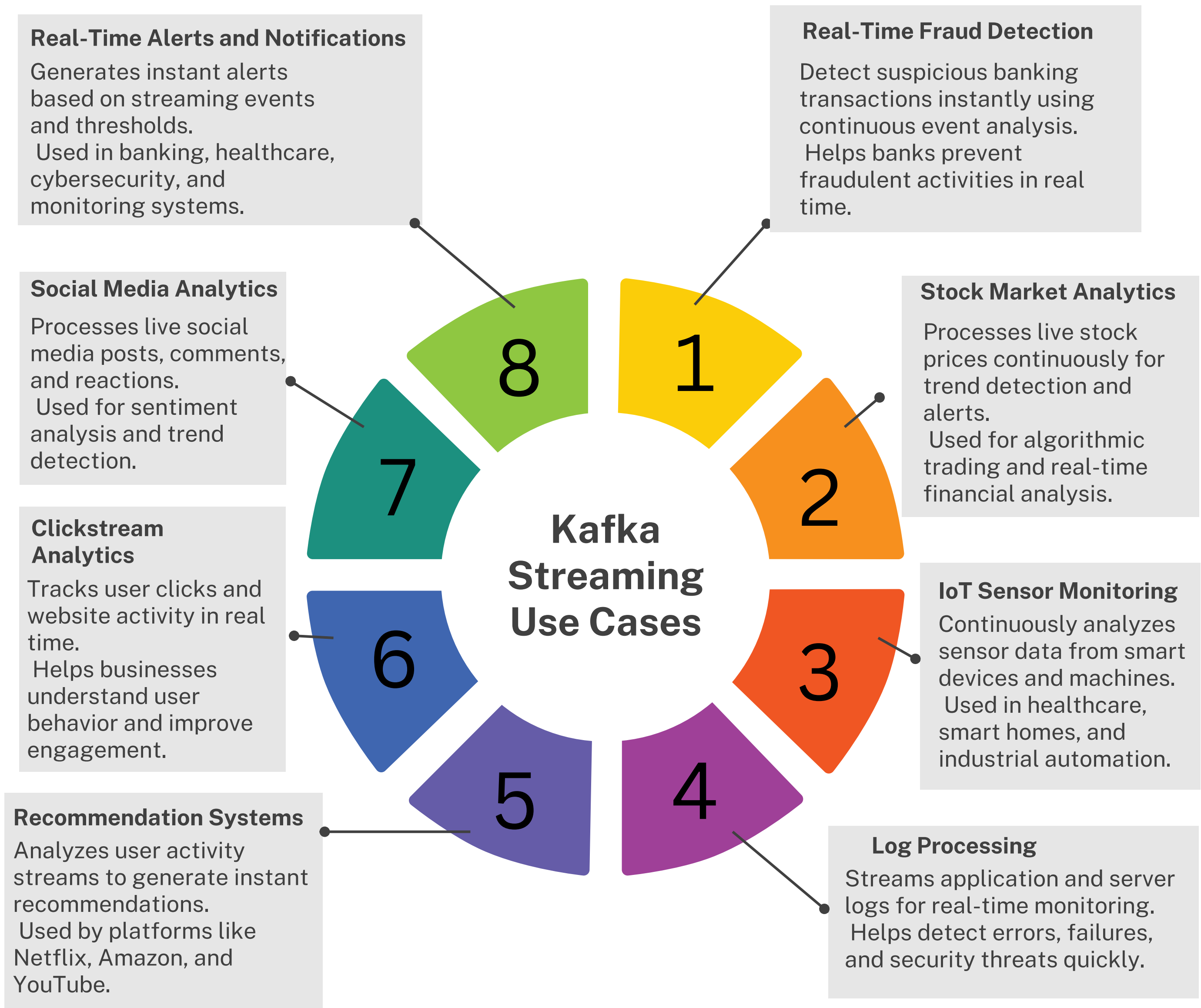
`stream.to("processed-stock-topic");`

## 8. CONSUMER READS PROCESSED DATA

Consumers or downstream applications read processed results.

Examples:

- Dashboards
- Alert systems





# Part VIII - Kafka Connect

## 8.1

### What is Kafka Connect

Apache Kafka Connect is a framework used to integrate Kafka with external systems like databases, cloud storage, search engines, and messaging systems. It enables scalable and fault-tolerant data movement without writing custom producer or consumer code.

Kafka Connect automates data integration between Kafka and other systems.

Kafka Connect is used for:

- Database integration
- Cloud storage integration
- Search engine integration
- Real-time ETL pipelines
- Data migration

#### Source Connector

Reads data from external systems and sends it to Kafka topics.

Example: MySQL → Kafka Topic

#### Sink Connector

Reads data from Kafka topics and sends it to external systems.

Example: Kafka Topic → Elasticsearch

#### Features

1. Scalability - Runs in distributed mode across multiple workers.
2. Fault Tolerance- Automatically recovers from failures.
3. Parallel Processing- Uses tasks for high-throughput data movement.
4. Easy Configuration- Connectors are configured using JSON or properties files.
5. No Custom Code- Reduces development effort significantly.

#### Benefits

- Simplifies integrations
- Supports real-time streaming
- Highly scalable
- Reliable data transfer
- Production-ready framework

Processed records are delivered into destination systems. Examples:

- Data warehouses
- Search engines
- Cloud storage
- Analytics platforms

Data is continuously generated from systems such as:

- Databases
- APIs
- Applications
- Sensors
- Cloud services

Sink connector consumes records from Kafka topics.

Common sink connectors:

- Elasticsearch Sink Connector
- S3 Sink Connector
- Snowflake Sink Connector

Extracted records are written into Kafka topics.

Example:  
orders-topic  
payments-topic  
customer-topic

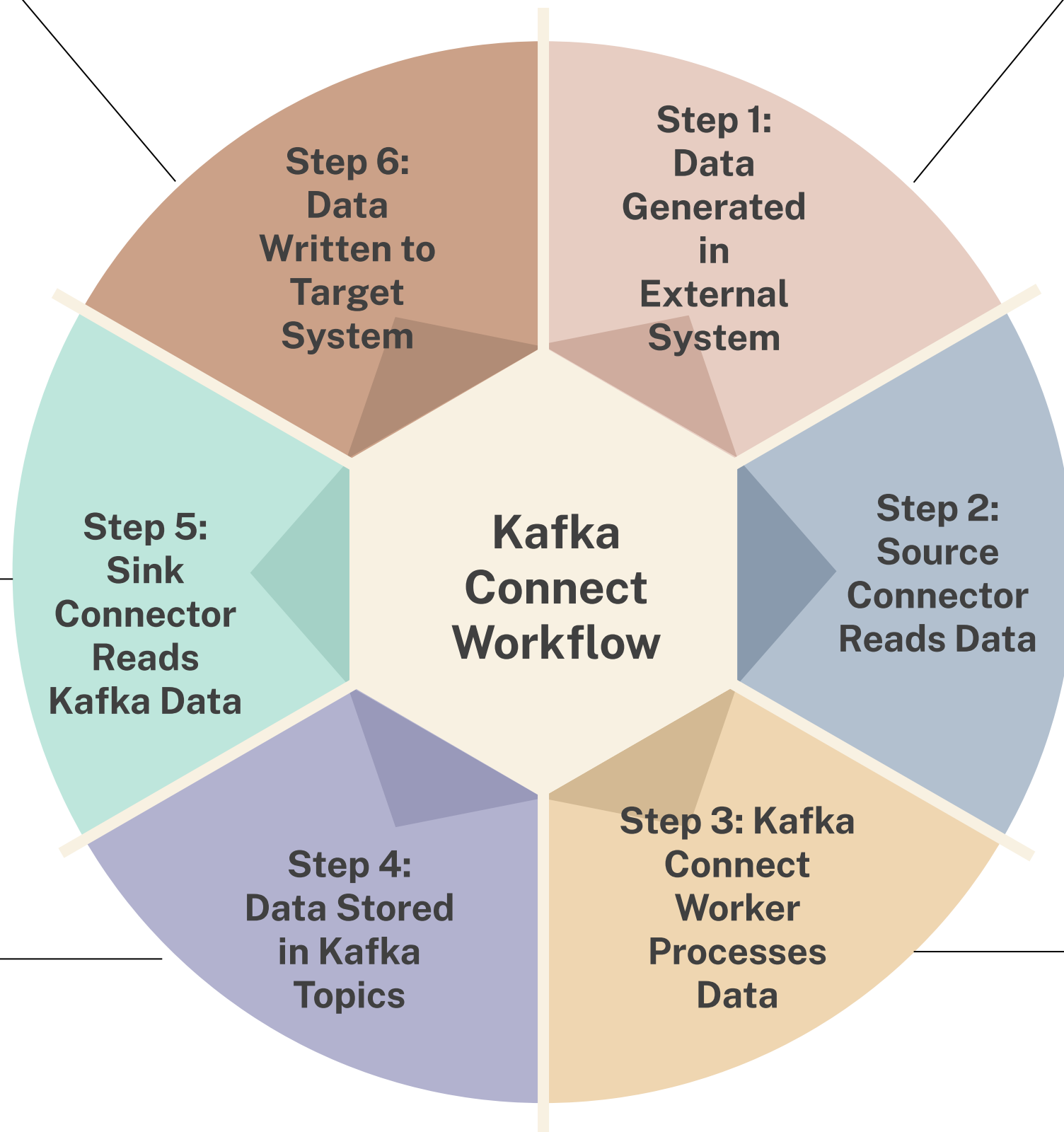
Kafka now acts as a central streaming platform.

Source connector extracts data from external systems. Common source connectors:

- JDBC Source Connector
- MongoDB Source Connector
- REST API Connector

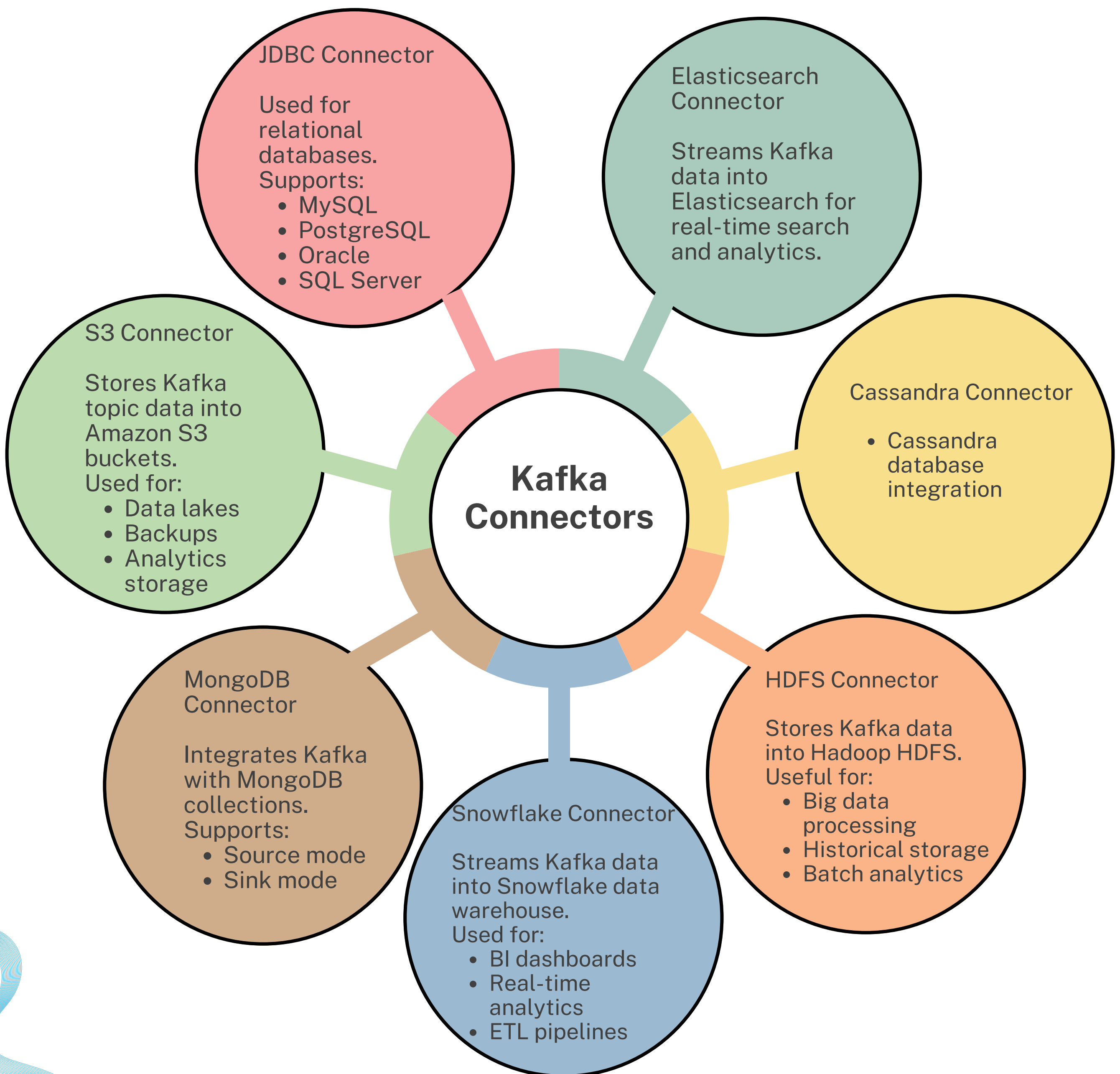
Kafka Connect workers manage connectors and tasks. Responsibilities:

- Data movement
- Fault tolerance
- Parallel execution
- Offset management



Kafka Connectors are ready-made plugins in Apache Kafka Connect used to transfer data between Kafka and external systems.

They eliminate the need to write custom producer or consumer applications.





# Part IX - Kafka Installation and Integration

## 1 Update Linux Packages

```
sudo apt update && sudo apt upgrade -y
```

## 2 Install Java (Required)

```
sudo apt install openjdk-23-jdk -y  
java -version
```

## 3 Install Kafka From Website

```
https://kafka.apache.org/community/downloads/
```

## 3 Extract Kafka

```
tar -xvf kafka_2.13-3.7.0.tgz  
cd kafka_2.13-3.7.0
```

## 4 Start Zookeeper

```
bin/zookeeper-server-start.sh config/zookeeper.properties
```

## 5 Start Kafka Broker

Open another terminal and start Kafka server:

```
bin/kafka-server-start.sh config/server.properties
```

## 6 Create Kafka Topic

```
bin/kafka-topics.sh --create \  
--topic test-topic \  
--bootstrap-server localhost:9092 \  
--partitions 1 \  
--replication-factor 1
```

## 7 List Topics

```
bin/kafka-topics.sh --list \  
--bootstrap-server localhost:9092
```

## 8 Start Producer

```
bin/kafka-console-producer.sh \  
--topic test-topic \  
--bootstrap-server localhost:9092
```

## 8 Start Consumer

```
bin/kafka-console-consumer.sh \  
--topic test-topic \  
--from-beginning \  
--bootstrap-server localhost:9092
```

# Kafka Integration with Tools

1

## Hadoop Integration

### Kafka Producer Code

```
from kafka import KafkaProducer
import json
import time

producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

for i in range(10):
    data = {
        "id": i,
        "name": f"user_{i}"
    }

    producer.send("users-topic", data)
    print("Sent:", data)

    time.sleep(1)

producer.close()
```



## Kafka Consumer + HDFS Write

```
from kafka import KafkaConsumer
from hdfs import InsecureClient
import json

consumer = KafkaConsumer(
    'users-topic',
    bootstrap_servers='localhost:9092',
    auto_offset_reset='earliest',
    value_deserializer=lambda x: json.loads(x.decode('utf-8'))
)

client = InsecureClient('http://localhost:9870',
user='hadoop')

hdfs_path = '/data/users/users_data.txt'

with client.write(hdfs_path, encoding='utf-8',
overwrite=True) as writer:

    for message in consumer:
        data = message.value

        writer.write(str(data) + '\\n')

    print("Written to HDFS:", data)
```

## Kafka Producer Code

```
from kafka import KafkaProducer
import json
import time

producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

for i in range(10):

    data = {
        "id": i,
        "amount": i * 100
    }

    producer.send("transactions", data)

    print("Sent:", data)

    time.sleep(1)

producer.close()
```

## Spark Structured Streaming Code

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

spark = SparkSession.builder \
    .appName("KafkaSparkStreaming") \
    .getOrCreate()

df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "transactions") \
    .load()

value_df = df.selectExpr("CAST(value AS STRING)")

query = value_df.writeStream \
    .outputMode("append") \
    .format("console") \
    .start()

query.awaitTermination()
```

## Submit Spark job with Kafka package:

```
spark-submit \
--packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0 \
kafka_spark.py
```



# X . Kafka Use Cases

